Introduction to Artificial Intelligence

1. What Is Artificial Intelligence?

Artificial Intelligence, commonly called AI, is the field of computer science focused on creating systems that can perform tasks that normally require human intelligence. These tasks can include understanding language, recognizing images, making predictions, solving problems, generating content, and making decisions based on information.

Traditional computer programs usually follow explicit instructions written by a programmer. For example, a programmer might create a program that says: if a customer's order is greater than $100, give the customer a 10% discount. The programmer has directly specified the rule.

AI systems can work differently. Instead of being given every possible rule, a machine learning system can be trained using examples. The system analyzes those examples and learns patterns that allow it to make predictions or decisions when it encounters new information.

Artificial Intelligence is therefore not one single technology. It is a broad field containing areas such as machine learning, deep learning, natural language processing, computer vision, reinforcement learning, generative AI, and information retrieval.

---

2. Artificial Intelligence and Machine Learning

Artificial Intelligence is the broader concept, while Machine Learning is one of the major approaches used to build AI systems.

Machine Learning allows computers to learn patterns from data instead of relying entirely on manually written rules.

Consider an email spam detector. Instead of writing thousands of rules describing every possible spam email, developers can provide the model with many examples of spam and legitimate emails. The machine learning algorithm analyzes these examples and learns patterns associated with spam.

After training, the model can receive a new email and estimate whether it is likely to be spam.

The basic process can be represented as:

Data → Training → Model → Prediction

The quality of the resulting model depends heavily on the quality and quantity of the training data, the learning algorithm, the model architecture, and the way the system is evaluated.

---

3. Deep Learning

Deep Learning is a branch of Machine Learning that uses neural networks containing multiple layers.

A neural network is a computational model inspired loosely by biological nervous systems. It contains interconnected units that process information and learn numerical parameters during training.

Deep neural networks can contain many layers. Earlier layers may learn relatively simple patterns while later layers can combine those patterns into more complex representations.

For example, an image recognition model might learn simple visual patterns such as edges in early layers. Later layers may combine those patterns into shapes, objects, and eventually complete concepts.

Deep learning has become particularly important because modern computers can process enormous datasets and perform large numbers of mathematical operations efficiently using hardware such as GPUs.

---

4. Large Language Models

A Large Language Model, or LLM, is a machine learning model designed to process and generate human language.

Modern LLMs are generally based on the Transformer architecture. They are trained on large collections of text and learn statistical patterns in language.

One important capability of an LLM is predicting the next token.

For example, given:

"The capital of France is"

the model may assign a high probability to the token "Paris".

The model does not simply store a database of sentences and retrieve one sentence every time. During training, it learns parameters that represent complex statistical relationships between tokens.

When generating text, the model repeatedly predicts what should come next based on the previous context.

The simplified process looks like:

Input text → Tokenization → Model → Next-token probabilities → Selected token → Repeat

This process happens repeatedly until the model produces the desired response.

---

5. Tokens

Language models do not normally process complete sentences as individual units. They process tokens.

A token can represent a complete word, part of a word, punctuation, or another piece of text.

For example, a sentence such as:

"Artificial intelligence is powerful."

may be divided into several tokens.

The exact tokenization depends on the tokenizer used by the model.

Tokens are important because model context windows, API pricing, memory requirements, and computational costs are often measured partly in tokens.

A longer prompt generally requires more tokens and therefore requires more computation.

---

6. Parameters

Parameters are numerical values learned by a machine learning model during training.

They can be thought of as adjustable values inside the model that help it represent patterns in the training data.

Large language models can contain billions of parameters.

A model with more parameters is not automatically better at every task. Performance also depends on the quality of the training data, model architecture, training process, context length, and other factors.

During training, the model repeatedly compares its predictions with expected results and adjusts its parameters to reduce the difference.

This process allows the model to gradually learn increasingly complicated patterns.

---

7. Training a Language Model

Training a language model requires large amounts of data and computational resources.

A simplified training process begins with a collection of text.

The text is converted into tokens. The model receives some tokens as context and attempts to predict another token.

For example:

"The cat is sitting on the"

The model might assign probabilities to possible next tokens such as:

chair
floor
table
bed

The training system knows the actual next token from the training data. It calculates how different the prediction was from the expected answer and uses that information to update the model's parameters.

This process happens many times across enormous amounts of data.

Over time, the model learns patterns involving grammar, vocabulary, facts, relationships between concepts, and many other characteristics of language.

---

8. Inference

Inference is the process of using a trained model to generate a prediction or response.

Training and inference are therefore different stages.

During training, the model's parameters are adjusted.

During inference, the parameters are generally fixed while the model uses them to process new input.

For a language model, inference may look like:

User question → Tokens → Model computation → Token prediction → Generated response

Inference can require significant computational resources, especially for large models.

This is one reason techniques such as quantization are useful for running language models locally.

---

9. Quantization

Quantization is a technique used to represent model weights using lower numerical precision.

A model may originally be stored using formats such as FP32 or FP16. Quantization can reduce the number of bits used to represent the model's values.

For example, a model may be quantized to 8-bit or 4-bit representations.

The major advantage is reduced memory usage and potentially faster inference on suitable hardware.

The trade-off is that reducing numerical precision can sometimes reduce model quality.

A 4-bit quantized model therefore attempts to provide a practical balance between model performance, memory requirements, and inference speed.

---

10. Retrieval-Augmented Generation

Retrieval-Augmented Generation, commonly called RAG, is a technique that combines information retrieval with language generation.

A normal language model answers questions using the knowledge represented in its parameters and the information contained in its current context.

However, a language model may not know the contents of a private company document, a recently created document, or a personal knowledge base.

RAG provides the model with relevant information from external documents before generating an answer.

A simplified RAG pipeline is:

User Question
↓
Retriever

↓
Relevant Document Chunks
↓
Prompt
↓
Language Model
↓
Answer

Instead of asking the language model to remember everything, the application retrieves useful information when it is needed.

---

11. Documents in a RAG System

A RAG system can work with many types of information.

Examples include:

PDF files
Text files
Web pages
Company documentation
Product manuals
Research papers
Customer support documents
Internal knowledge bases
Database records

Before the information can be retrieved efficiently, it usually needs to be loaded and processed.

A document loader is responsible for reading information from a particular source and converting it into a format that the application can work with.

For example, a PDF loader can read a PDF and produce document objects containing text and metadata.

---

12. Text Splitting

Large documents are usually divided into smaller pieces called chunks.

This process is called text splitting or chunking.

Suppose a 100-page document is given to a RAG system. Sending the entire document to the language model every time a user asks a question would be inefficient.

Instead, the document can be divided into smaller chunks.

For example:

Document
↓
Chunk 1
Chunk 2
Chunk 3
Chunk 4
Chunk 5

When the user asks a question, the system can search these chunks and retrieve only the pieces that are relevant.

Chunk size is an important design decision.

If chunks are extremely small, important context may be lost.

If chunks are extremely large, retrieval may return too much irrelevant information.

Many RAG systems therefore use a combination of chunk size and chunk overlap.

Chunk overlap means that a small amount of text is repeated between neighboring chunks so that information near a chunk boundary is less likely to lose context.

---

13. Embeddings

An embedding is a numerical representation of information.

An embedding model converts text into a vector of numbers.

For example, the sentence:

"Python is a programming language."

could be converted into a vector such as:

[0.21, -0.44, 0.87, 0.13, ...]

The actual vector contains many more dimensions than this simplified example.

The important idea is that text with similar meanings can have embeddings that are close to one another in vector space.

For example:

"How do I learn Python?"

and

"What is the best way to study Python programming?"

have different words but similar meanings.

A good embedding model should represent that semantic similarity.

---

14. Vector Stores

A vector store is a system designed to store and search vector representations.

After documents are split into chunks, an embedding model converts each chunk into a vector.

The vector and its associated document information can then be stored in a vector database.

The simplified process is:

```
Document
↓
Chunks
↓
Embedding Model
↓
Vectors
↓
Vector Store
```

When a user asks a question, the question is also converted into an embedding.

The vector store compares the question vector with stored document vectors and identifies chunks that are mathematically similar.

---

15. Similarity Search

Similarity search attempts to find information that is semantically related to a query.

Suppose a knowledge base contains information about Python, PostgreSQL, artificial intelligence, and networking.

A user asks:

"What is a Python function?"

The system creates an embedding for the question.

It then compares that embedding with the embeddings of stored document chunks.

Chunks discussing Python functions should receive high similarity scores, while unrelated chunks about networking should receive lower scores.

The retriever can then return the most relevant chunks.

---

16. The Retriever

A retriever is the component responsible for finding relevant information from a knowledge source.

In a RAG system, the retriever usually receives a user query and returns relevant document chunks.

For example:

```
Question:
"What are parameters in a language model?"

Retriever:
Chunk 17
Chunk 42
Chunk 53
```

These chunks are then provided to the language model.

The retriever therefore acts as a bridge between the user's question and the external knowledge base.

---

17. Prompt Construction in RAG

After retrieving relevant chunks, the application needs to construct a prompt for the language model.

A simplified prompt might look like:

```
Context:
Parameters are numerical values learned by a machine learning model during training...

Question:
What are parameters?

Instruction:
Answer the question using the provided context.
```

The language model receives this information and generates an answer.

The quality of the final answer depends on several components, including the quality of the documents, chunking strategy, embedding model, retrieval strategy, prompt design, and language model.

---

18. Why RAG Is Useful

RAG is useful when an application needs to answer questions using a specific external knowledge source.

For example, a company could create a RAG assistant that understands its employee handbook.

An employee could ask:

"How many vacation days do I receive?"

The system could retrieve the relevant section of the handbook and provide it to the language model.

The model can then generate a natural-language answer based on that retrieved information.

RAG can therefore be useful for customer support, internal documentation, research assistants, educational applications, technical documentation, and personal knowledge bases.

---

19. RAG Versus Fine-Tuning

RAG and fine-tuning solve different problems.

RAG provides external information to a model at inference time.

Fine-tuning changes the model's parameters by training it further on a specialized dataset.

If the goal is to make a model answer questions using a changing collection of company documents, RAG can be useful because documents can be updated without retraining the language model.

If the goal is to change how a model behaves, follows a particular style, or performs a specialized task, fine-tuning may be considered.

The two approaches can also be combined.

---

20. Local Language Models

Language models do not always need to run through a cloud API.

A model can sometimes be downloaded and executed locally on a personal computer.

Running a model locally can provide benefits such as greater control over data, offline operation, and avoiding per-request API charges.

However, local inference requires suitable hardware.

Memory requirements depend on the model architecture, model size, numerical precision, context length, and inference software.

Quantized models make local inference more accessible because they require less memory than their higher-precision versions.

---

21. Qwen and GGUF

Qwen is a family of language models developed by Alibaba's Qwen team.

Models can be distributed in different formats depending on the software used to run them.

GGUF is a model file format commonly used with the llama.cpp ecosystem.

A GGUF file can contain the model weights and related information required by compatible inference software.

A quantized Qwen model such as:

Qwen3-4B-Q4_K_M.gguf

represents a Qwen model with approximately 4 billion parameters using a particular 4-bit quantization format.

The exact memory and performance characteristics depend on the implementation and hardware.

---

22. A Complete Local RAG Architecture

A local RAG application can be organized into several components.

The first component loads documents.

The second component splits documents into smaller chunks.

The third component creates embeddings for those chunks.

The fourth component stores the embeddings inside a vector database.

The fifth component retrieves relevant chunks when a question is asked.

The sixth component sends the retrieved context and user question to a local language model.

The final component displays the generated answer.

The complete architecture can therefore be represented as:

```
Documents
↓
Document Loader
↓
Text Splitter
↓
Embedding Model
↓
Vector Store
↓
Retriever
↓
Relevant Context
↓
Prompt
↓
Qwen Language Model
↓
Generated Answer
```

---

23. Example RAG Question

Imagine the knowledge base contains this statement:

"Quantization reduces the numerical precision used to represent model weights and can reduce memory requirements."

The user asks:

"Why would someone quantize a language model?"

The retriever searches the vector store and finds the relevant chunk.

That chunk is placed into the prompt.

The Qwen model receives:

```
Context:
Quantization reduces the numerical precision used to represent model weights
and can reduce memory requirements.

Question:
Why would someone quantize a language model?
```

The model can then generate an answer such as:

"Quantization can make a language model require less memory, which can make it easier to run locally on hardware with limited resources."

The important point is that the answer is generated using retrieved external context.

---

24. Limitations of RAG

RAG does not automatically guarantee correct answers.

If the document contains incorrect information, the system may retrieve that incorrect information.

If the text splitter creates poor chunks, important context may be lost.

If the embedding model fails to represent the meaning accurately, the retriever may return irrelevant information.

If the retriever returns the wrong chunks, the language model may not have the information required to answer correctly.

The language model itself can also generate incorrect statements.

Therefore, building a reliable RAG system requires testing every stage of the pipeline.

A useful RAG evaluation process can examine:

1. Document quality
2. Chunk quality
3. Embedding quality
4. Retrieval quality
5. Prompt quality
6. Generated answer quality

---

25. Summary

Artificial Intelligence is a broad field involving systems capable of performing tasks associated with human intelligence.

Machine Learning allows systems to learn patterns from data.

Deep Learning uses multi-layer neural networks.

Large Language Models process and generate language using learned parameters.

Tokens are the units processed by language models.

Parameters are numerical values learned during training.

Quantization reduces numerical precision to make models more efficient.

Retrieval-Augmented Generation combines information retrieval with language generation.

A RAG system can load documents, split them into chunks, convert those chunks into embeddings, store them in a vector database, retrieve relevant chunks, and provide those chunks to a language model.

A local Qwen model can then use the retrieved context to generate an answer.

The fundamental RAG pipeline is:

Documents
→ Document Loader
→ Text Splitter
→ Embeddings

→ Vector Store
→ Retriever
→ Context
→ Qwen
→ Answer